CSEE 4121 — Practice Midterm

Computer Systems for Data Science | Spring 2026

- **Time:** 90 minutes
- **Questions:** 20 (mix of multiple choice, short answer, and SQL)
- **Total Points:** 100
- **No cheat sheet allowed**

1 PERFORMANCE 5 pts

A program spends 40% of its time in a component that can be parallelized. If you speed up that component by 4x, what is the overall speedup according to Amdahl's Law?

- (●) a) 1.18x
- ( ) b) 1.43x
- ( ) c) 2.50x
- ( ) d) 4.00x

> **Incorrect (you picked: A, correct: B).** Amdahl's Law: Speedup = 1 / ((1 - 0.4) + 0.4/4) = 1 / (0.6 + 0.1) = 1 / 0.7 = 1.43x.

2 PERFORMANCE 5 pts

Rank the following from fastest to slowest access latency:

- (●) a) L1 cache < L2 cache < Main memory < SSD < HDD
- ( ) b) L2 cache < L1 cache < SSD < Main memory < HDD
- ( ) c) L1 cache < Main memory < L2 cache < HDD < SSD
- ( ) d) L1 cache < L2 cache < SSD < Main memory < HDD

> **Correct!** The memory hierarchy from fastest to slowest: L1 cache (~1ns) < L2 cache (~4ns) < Main memory (~100ns) < SSD (~100μs) < HDD (~10ms).

3 RELATIONAL MODEL 5 pts

What is the difference between a set and a multiset (bag) in the context of relational databases?

- ( ) a) A set allows duplicates; a multiset does not
- (●) b) A set does not allow duplicates; a multiset allows duplicates
- ( ) c) A set is ordered; a multiset is unordered
- ( ) d) There is no difference; they are synonyms

> **Correct!** A set has no duplicates (like a mathematical set). A multiset (bag) allows duplicates. SQL tables are multisets by default — you need SELECT DISTINCT to get set behavior.

4 SQL 5 pts

Given a table `orders(id, customer_id, amount, order_date)`, write a SQL query to find each customer's total spending, but only for customers who have spent more than $1000.

```
Write your SQL here...
```

> **No answer provided.**

Model answer:

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 1000;
```

You need GROUP BY customer_id to aggregate per customer, SUM(amount) for total spending, and HAVING (not WHERE) to filter on the aggregate.

5 SQL / JOINS 5 pts

Table `students(id, name)` has 5 rows. Table `enrollments(student_id, course)` has 8 rows. How many rows does `SELECT * FROM students CROSS JOIN enrollments` return?

- ○ a) 8
- ○ b) 13
- ◉ c) 40
- ○ d) 5

Correct! CROSS JOIN produces the Cartesian product: 5 × 8 = 40 rows. Every row from students is paired with every row from enrollments.

6 SQL / JOINS 5 pts

You have `employees(id, name, dept_id)` and `departments(id, dept_name)`. Some employees have a NULL `dept_id`. Which JOIN guarantees all employees appear in the result, even those without a department?

- ○ a) INNER JOIN
- ◉ b) LEFT JOIN employees ON departments
- ○ c) RIGHT JOIN employees ON departments
- ○ d) CROSS JOIN

Correct! LEFT JOIN keeps all rows from the left table (employees), filling in NULLs where there's no matching department. INNER JOIN would drop employees without departments.

7 PROTOCOL BUFFERS 5 pts

Why are Protocol Buffers (protobuf) smaller than JSON for the same data?

- ○ a) Protobuf compresses data using gzip
- ◉ b) Protobuf uses numeric field tags instead of string keys and a binary encoding
- ○ c) Protobuf removes all metadata
- ○ d) Protobuf stores only primary keys

Correct! Protobuf replaces human-readable string field names with small numeric tags (1, 2, 3...) and uses binary variable-length encoding instead of text. This makes it much more compact than JSON.

8 TRANSACTIONS / ACID 5 pts

Which ACID property does Write-Ahead Logging (WAL) primarily provide?

- ○ a) Consistency and Isolation
- ◉ b) Atomicity and Durability
- ○ c) Only Isolation
- ○ d) Only Consistency

Correct! WAL provides Atomicity (can undo incomplete transactions by replaying the log) and Durability (changes are logged to disk before being applied, surviving crashes).

9 ISOLATION LEVELS 5 pts

Transaction T1 reads a row, then transaction T2 modifies and commits that row, then T1 reads the same row again and sees the new value. What anomaly is this?

- ○ a) Dirty read
- ◉ b) Unrepeatable read
- ○ c) Phantom read
- ○ d) Lost update

> **Correct!** Unrepeatable read: T1 reads a row, T2 commits a change to that row, T1 re-reads and sees a different value. A dirty read would be if T2 hadn't committed yet.

10 CONCURRENCY / LOCKING 5 pts

In Two-Phase Locking (2PL), what is the rule that guarantees conflict serializability?

- ○ a) A transaction must acquire all locks before executing any operation
- ◉ b) Once a transaction releases any lock, it cannot acquire any new locks
- ○ c) All shared locks must be acquired before exclusive locks
- ○ d) Locks are released in the reverse order of acquisition

> **Correct!** The 2PL rule: there's a growing phase (acquire locks) and a shrinking phase (release locks). Once any lock is released, no new locks can be acquired. This guarantees conflict serializability.

11 SERIALIZABILITY 5 pts

Given this schedule with transactions T1 and T2:

```
T1: R(A)      W(A)           R(B)  W(B)
T2:      R(A)      W(A)  R(B)           W(B)
```

List all conflicts in this schedule (e.g., "T1:R(A)-T2:W(A)"). Is this schedule conflict-serializable?

```
List the conflicts and state whether it is conflict-serializable...
```

> **No answer provided.**
>
> **Model answer:**
>
> ```
> Conflicts (operations on same item, at least one write, different
> transactions):
> 1. T1:R(A) - T2:W(A)  (RW conflict on A)
> 2. T2:R(A) - T1:W(A)  (RW conflict on A)
> 3. T1:W(A) - T2:W(A)  (WW conflict on A)
> 4. T2:R(B) - T1:W(B)  (RW conflict on B)
> 5. T1:R(B) - T2:W(B)  (RW conflict on B)
> 6. T1:W(B) - T2:W(B)  (WW conflict on B)
>
> Conflict graph: T1 → T2 (from conflict 1) and T2 → T1 (from conflict 2).
> There is a cycle, so this schedule is NOT conflict-serializable.
> ```
>
> Build a conflict graph from the conflicting operations. If the graph has a cycle, the schedule is not conflict-serializable. Here T1→T2 and T2→T1 creates a cycle.

12 LOCKING 5 pts

Two transactions: T1 holds an exclusive lock on row X and requests a shared lock on row Y. T2 holds an exclusive lock on row Y and requests a shared lock on row X. What happens?

- ○ a) Both proceed normally
- ◉ b) Deadlock — each is waiting for the other
- ○ c) T1 is aborted, T2 proceeds

○ d) Both are upgraded to exclusive locks automatically

> **Correct!** Classic deadlock: T1 waits for Y (held by T2), T2 waits for X (held by T1). Neither can proceed. The DBMS must detect this (via a waits-for graph) and abort one transaction.

13 INDEXING 5 pts

You have a query: `SELECT * FROM users WHERE LOWER(email) = 'foo@bar.com'`. There is an index on the `email` column. Will the database use the index? Why or why not?

```
Your answer...
```

> **No answer provided.**
>
> **Model answer:**
>
> ```
> No, the database will NOT use the index. Applying a function (LOWER()) to the column prevents the optimizer from using the index on email. The index stores raw email values, but the query compares against LOWER(email), which doesn't match the index structure. Fix: create a functional index on LOWER(email), or store emails in lowercase.
> ```
>
> Functions applied to indexed columns are index inhibitors. The index stores original values, so LOWER(email) can't use it. This forces a full table scan.

14 B+ TREES 5 pts

Why are B+ Trees preferred over binary search trees for database indexing?

○ a) B+ Trees use less memory in all cases

○ b) B+ Trees have high fan-out, which reduces the number of disk I/Os needed for a lookup

○ c) Binary search trees do not support range queries

○ d) B+ Trees are always faster than hash indexes

> **Incorrect (you picked: none, correct: B).** B+ Trees have a high fan-out (hundreds of children per node) so the tree is very shallow (3-4 levels for millions of keys). Each level is a disk I/O, so fewer levels = fewer disk reads. BSTs have fan-out 2, making them much deeper.

15 BLOOM FILTERS 5 pts

A Bloom filter returns "possibly in set" for a key lookup. Which of the following is true?

○ a) The key is definitely in the set

◉ b) The key might or might not be in the set (false positive possible)

○ c) The Bloom filter is broken and should be rebuilt

○ d) The key was in the set but has since been deleted

> **Correct!** Bloom filters have no false negatives (if it says "not in set," it's definitely not). But they can have false positives — "possibly in set" means the key might or might not actually be there.

16 CACHING 5 pts

A cache with 3 slots processes the sequence: `A, B, C, A, D, B` using LRU. What is evicted when D arrives?

○ a) A

◉ b) B

○ c) C

○ d) Nothing — cache is not full

> **Correct!** Trace: A miss [A], B miss [A,B], C miss [A,B,C], A hit (moved to MRU) [B,C,A], D arrives — cache full, evict LRU which is B. Cache becomes [C,A,D].

17 NETWORKING 5 pts

Match each item to its correct network layer:

MAC address: Data Link

Port number: Transport

IP address: Network

HTTP: Application

All correct! MAC → Data Link, Port → Transport, IP → Network, HTTP → Application.

18 NETWORKING 5 pts

When a packet travels through 10 routers from your laptop to a server, which addresses **change at every hop** and which **stay the same**?

- (●) a) IP addresses change; MAC addresses stay the same
- ( ) b) Both change at every hop
- ( ) c) MAC addresses change at every hop; IP addresses stay the same
- ( ) d) Neither changes

Incorrect (you picked: A, correct: C). IP addresses (Layer 3) identify the source and destination globally and stay the same end-to-end. MAC addresses (Layer 2) are local to each hop — they change at every router because they identify the next device on the link.

19 DISTRIBUTED SYSTEMS 5 pts

In Two-Phase Commit (2PC), if **one** participant votes NO in the Prepare phase, what does the coordinator do?

- ( ) a) Commits anyway with the remaining participants
- (●) b) Sends ABORT to all participants
- ( ) c) Retries the Prepare phase up to 3 times
- ( ) d) Asks the NO voter to change its vote

Correct! 2PC requires unanimous YES votes to commit. If any participant votes NO, the coordinator sends ABORT to all participants. The transaction is rolled back everywhere.

20 PARTITIONING 5 pts

What problem does **consistent hashing** solve compared to regular hash partitioning (key mod N)?

- ( ) a) It eliminates hash collisions entirely
- (●) b) When a node is added/removed, only a small fraction of keys need to be moved instead of rehashing everything
- ( ) c) It ensures all partitions have exactly equal data
- ( ) d) It makes range queries faster

Correct! With key mod N, adding or removing a node changes N, causing almost every key to rehash to a different node. Consistent hashing uses a ring, so only the keys on the affected segment of the ring need to move — roughly 1/N of the data.

Grade Exam Reset

70 / 100

70% — Decent — focus on the red questions.

CSEE 4121 Practice Midterm — Columbia University — Spring 2026